

LAPORAN PRAKTIKUM
STRUKTUR DATA
SINGLE LINKED LIST



Disusun oleh:

Ghinada Fathanawafa Algma
2511533008

Dosen Pengampu:
Dr. Wahyudi S.T. M.T

Asisten Praktikum:
Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa, berkat Rahmat dan karunia-Nya laporan praktikum Struktur Data tentang Double Linked List menggunakan bahasa Java ini dapat saya selesaikan dengan baik. Laporan ini saya susun sebagai bentuk pertanggungjawaban atas praktikum yang telah saya laksanakan.

Dalam penyusunan laporan ini saya banyak mendapat bimbingan serta dukungan dari beberapa pihak, oleh karena itu saya mengucapkan terima kasih kepada dosen pengampu, asisten praktikum dan rekan rekan praktikan yang turut membantu dalam pelaksanaan kegiatan praktikum.

Saya menyadari bahwa laporan praktikum ini jauh dari sempurna. Oleh karena itu kritik dan saran yang membangun sangat berguna untuk perkembangan lebih baik kedepannya. Semoga laporan praktikum ini dapat bermanfaat bagi saya serta pembaca, khususnya dalam memahami konsep Double Linked List.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori	2
2.1.1 Konsep Dasar	2
2.1.2 Operasi Utama DLL	2
2.1.3 Kelebihan dan Kekurangan DLL	2
2.2 Kode Program	3
2.2.1 Class NodeDLL_2511533008.....	3
2.2.2 Class InsertDLL_2511533008	4
2.2.3 Class HapusDLL_2511533008	8
2.2.4 Class PenelusuranDLL_2511533008	12
2.3 Output Program	15
2.3.1 Output InsertDLL_2511533008.....	15
2.3.2 Output HapusDLL_2511533008.....	15
2.3.3 Output PenelusuranDLL_2511533008	15
BAB III PENUTUP	16
3.1 Kesimpulan	16

DAFTAR PUSTAKA	17
TUGAS	18
1. Kode Program	18
1.1 Class Lagu_2511533008	18
1.2 Class Musik_2511533008	18
2. Output Tugas	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Salah satu struktur data yang penting adalah Double Linked List (DLL), yaitu kumpulan simpul (node) yang saling terhubung secara berurutan melalui dua pointer: *next* dan *prev*. Berbeda dengan Single Linked List yang hanya memiliki satu arah keterhubungan, Double Linked List memungkinkan traversal dilakukan baik dari depan ke belakang maupun sebaliknya. Hal ini memberikan fleksibilitas lebih dalam operasi seperti penambahan, penghapusan, maupun pencarian data. Melalui praktikum ini, kita diharapkan mampu memahami konsep dasar node, head, tail, serta implementasi operasi dasar pada DLL menggunakan bahasa pemrograman seperti Java. Penguasaan terhadap Double Linked List tidak hanya memperkuat logika pemrograman, tetapi juga menjadi landasan untuk mempelajari struktur data yang lebih kompleks seperti tree dan graph.

1.2 Tujuan

1. Memahami konsep dasar Double Linked List (DLL) sebagai salah satu struktur data.
2. Mengenali komponen utama linked list: node, head, dan pointer.
3. Mempelajari dan mengimplementasikan operasi dasar: traversal, insertion, deletion, dan searching.

1.3 Manfaat

1. Memahami konsep dasar Double Linked List (DLL) sebagai salah satu struktur data.
2. Dapat mengenali komponen utama linked list: node, head, dan pointer.
3. Bisa mengimplementasikan operasi dasar: traversal, insertion, deletion, dan searching

BAB II

PEMBAHASAN

2.1 Dasar Teori

2.1.1 Konsep Dasar

DLL merupakan struktur data linear yang terdiri dari simpul (node) yang saling terhubung 2 arah.

Node : setiap simpul memiliki 3 bagian

1. Data → menyimpan nilai/informasi.
2. Pointer next → menunjuk ke simpul berikutnya.
3. Pointer prev → menunjuk ke simpul sebelumnya.

Head : simpul pertama dalam list.

Tail : simpul terakhir, pointer next-nya bernilai null.

2.1.2 Operasi Utama DLL

1. Traversal (Penelusuran)

Bisa dilakukan dari head → tail atau sebaliknya dari tail → head.

2. Insertion (Penyisipan)

- a. Di depan : node baru menjadi head.
- b. Di belakang : node baru ditambahkan setelah tail.
- c. Di tengah : node baru disisipkan diantara 2 node, dengan mengatur prev dan next.

3. Deletion (Penghapusan)

- a. Head : menghapus simpul pertama
- b. Tail : menghapus simpul terakhir
- c. Posisi tertentu : menghapus simpul pada Lokasi tertentu dengan menyesuaikan pointer prev dan next.

2.1.3 Kelebihan dan Kekurangan DLL

1. Kelebihan

- a. Traversal dua arah (lebih fleksibel).
 - b. Operasi insert/delete lebih mudah karena ada pointer prev.
2. Kekurangan
- a. Membutuhkan memori lebih besar.
 - b. Implementasi lebih kompleks dibanding SLL.

2.2 Kode Program

2.2.1 Class NodeDLL_2511533008

Kode Program:

```
package pekan6_2511533008;

public class NodeDLL_2511533008 {
    //mendefinisikan kelas node
    int data_3008;
    NodeDLL_2511533008 next_3008; //pointer ke next node
    NodeDLL_2511533008 prev_3008; //pointer ke previous node

    //konstruktor
    public NodeDLL_2511533008(int data_3008) {
        this.data_3008 = data_3008;
        this.next_3008 = null;
        this.prev_3008 = null;
    }
}
```

Penjelasan:

1. Atribut data_3008 berupa integer
2. Pointer

```
int data_3008;
NodeDLL_2511533008 next_3008; //pointer ke next node
NodeDLL_2511533008 prev_3008; //pointer ke previous node
```

Gambar 2.2.1.1 pointer

3. Konstruktor

```
//konstruktor
public NodeDLL_2511533008(int data_3008) {
    this.data_3008 = data_3008;
    this.next_3008 = null;
    this.prev_3008 = null;
}
```

Gambar 2.2.1.2 konstruktor

2.2.2 Class InsertDLL_2511533008

Kode Program:

```
package pekan6_2511533008;

public class InsertDLL_2511533008 {
    //menambahkan node di awal dll
    static NodeDLL_2511533008 insertBegin_3008(NodeDLL_2511533008
head_3008, int data_3008) {
        //buat node baru
        NodeDLL_2511533008 new_node_3008 = new
NodeDLL_2511533008(data_3008);
        //jadikan pointer nextnya head
        new_node_3008.next_3008 = head_3008;
        //jadikan pointer prev head ke new_node_3008
        if (head_3008 != null) {
            head_3008.prev_3008 = new_node_3008;
        }
        return new_node_3008;
    }
    //fungsi menambahkan node di akhir
    public static NodeDLL_2511533008
insertEnd_3008(NodeDLL_2511533008 head_3008, int newData_3008) {
        //buat node baru
        NodeDLL_2511533008 newNode_3008 = new
NodeDLL_2511533008(newData_3008);
        //jika dll null jadikan head
        if (head_3008 == null) {
            head_3008 = newNode_3008;
        }
        else {
            NodeDLL_2511533008 curr_3008 = head_3008;
            while (curr_3008.next_3008 != null) {
                curr_3008 = curr_3008.next_3008;
            }
            curr_3008.next_3008 = newNode_3008;
            newNode_3008.prev_3008 = curr_3008;
        }
        return head_3008;
    }
    //fungsi menambahkan node di posisi tertentu
    public static NodeDLL_2511533008
insertAtPosition_3008(NodeDLL_2511533008 head_3008, int pos_3008,
int new_data_3008) {
        //buat node baru
        NodeDLL_2511533008 new_node_3008 = new
NodeDLL_2511533008(new_data_3008);
        if (pos_3008 == 1) {
            new_node_3008.next_3008 = head_3008;
            if (head_3008 != null) {
                head_3008.prev_3008 = new_node_3008;
            } head_3008 = new_node_3008;
        }
```



```

        return head_3008;
    }
    NodeDLL_2511533008 curr_3008 = head_3008;
    for (int i_3008 = 1; i_3008 < pos_3008 - 1 && curr_3008
!= null; ++i_3008) {
        curr_3008 = curr_3008.next_3008;
    }
    if (curr_3008 == null) {
        System.out.println("Posisi tidak ada");
        return head_3008;
    }
    new_node_3008.prev_3008 = curr_3008;
    new_node_3008.next_3008 = curr_3008.next_3008;
    curr_3008.next_3008 = new_node_3008;
    if (new_node_3008.next_3008 != null) {
        new_node_3008.next_3008.prev_3008 =
new_node_3008;
    } return head_3008;
}

public static void printList_3008(NodeDLL_2511533008
head_3008) {
    NodeDLL_2511533008 curr_3008 = head_3008;
    while (curr_3008 != null) {
        System.out.print(curr_3008.data_3008 + " <-> ");
        curr_3008 = curr_3008.next_3008;
    }
    System.out.println();
}

public static void main(String[] args) {
    //membuat dll 2 <-> 3 <-> 5
    NodeDLL_2511533008 head_3008 = new
NodeDLL_2511533008(2);
    head_3008.next_3008 = new NodeDLL_2511533008(3);
    head_3008.next_3008.prev_3008 = head_3008;
    head_3008.next_3008.next_3008 = new
NodeDLL_2511533008(5);
    head_3008.next_3008.next_3008.prev_3008 =
head_3008.next_3008;
    //cetak dll awal
    System.out.print("DLL awal: ");
    printList_3008(head_3008);
    //tambah 1 diawal
    head_3008 = insertBegin_3008(head_3008, 1);
    System.out.print("simpul 1 ditambah di awal: ");
    printList_3008(head_3008);
    //tambah 6 diakhir
    System.out.print("simpul 6 ditambah di akhir: ");
    int data_3008 = 6;
    head_3008 = insertEnd_3008(head_3008, data_3008);
    printList_3008(head_3008);
    //menambah node 4 di posisi 4
    System.out.print("tambah node 4 di posisi 4: ");

```

```

        int data2_3008 = 4;
        int pos_3008 = 4;
        head_3008 = insertAtPosition_3008(head_3008, pos_3008,
data2_3008);
        printList_3008(head_3008);

    }
}

```

Penjelasan:

1. Fungsi menambah node di awal

```

//menambahkan node di awal dll
static NodeDLL_2511533008 insertBegin_3008(NodeDLL_2511533008 head_3008, int data_3008) {
    //buat node baru
    NodeDLL_2511533008 new_node_3008 = new NodeDLL_2511533008(data_3008);
    //jadikan pointer nextnya head
    new_node_3008.next_3008 = head_3008;
    //jadikan pointer prev head ke new_node_3008
    if (head_3008 != null) {
        head_3008.prev_3008 = new_node_3008;
    }
    return new_node_3008;
}

```

Gambar 2.2.2.1 fungsi insert awal

Pada fungsi ini akan membuat node baru lalu menjadikan node baru sebagai head dan jika head lama ada maka pointer prev diarahkan ke head baru.

2. Fungsi menambah node di akhir

```

//fungsi menambahkan node di akhir
public static NodeDLL_2511533008 insertEnd_3008(NodeDLL_2511533008 head_3008, int newData_3008) {
    //buat node baru
    NodeDLL_2511533008 newNode_3008 = new NodeDLL_2511533008(newData_3008);
    //jika dll null jadikan head
    if (head_3008 == null) {
        head_3008 = newNode_3008;
    }
    else {
        NodeDLL_2511533008 curr_3008 = head_3008;
        while (curr_3008.next_3008 != null) {
            curr_3008 = curr_3008.next_3008;
        }
        curr_3008.next_3008 = newNode_3008;
        newNode_3008.prev_3008 = curr_3008;
    }
    return head_3008;
}

```

Gambar 2.2.2.2 fungsi insert akhir

Jika list kosong, node baru akan jadi head. Jika tidak kosong, traversal sampai node terakhir, lalu sambungkan node baru di akhir.

3. Fungsi menambah node pada posisi tertentu

```
//fungsi menambahkan node di posisi tertentu
public static NodeDLL_2511533008 insertAtPosition_3008(NodeDLL_2511533008 head_3008, int pos_3008, int new_data_3008) {
    //buat node baru
    NodeDLL_2511533008 new_node_3008 = new NodeDLL_2511533008(new_data_3008);
    if (pos_3008 == 1) {
        new_node_3008.next_3008 = head_3008;
        if (head_3008 != null) {
            head_3008.prev_3008 = new_node_3008;
        }
        head_3008 = new_node_3008;
        return head_3008;
    }
    NodeDLL_2511533008 curr_3008 = head_3008;
    for (int i_3008 = 1; i_3008 < pos_3008 - 1 && curr_3008 != null; ++i_3008) {
        curr_3008 = curr_3008.next_3008;
    }
    if (curr_3008 == null) {
        System.out.println("Posisi tidak ada");
        return head_3008;
    }
    new_node_3008.prev_3008 = curr_3008;
    new_node_3008.next_3008 = curr_3008.next_3008;
    curr_3008.next_3008 = new_node_3008;
    if (new_node_3008.next_3008 != null) {
        new_node_3008.next_3008.prev_3008 = new_node_3008;
    }
    return head_3008;
}
```

Gambar 2.2.2.3 insert node di posisi tertentu

Jika posisi = 1, node baru jadi head. Jika posisi lain, traversal sampai node sebelum posisi lalu sambungkan node baru di posisi tersebut. Dan jika posisi tidak valid, akan menampilkan pesan posisi tidak ada.

4. Fungsi cetak list

```
public static void printList_3008(NodeDLL_2511533008 head_3008) {
    NodeDLL_2511533008 curr_3008 = head_3008;
    while (curr_3008 != null) {
        System.out.print(curr_3008.data_3008 + " <-> ");
        curr_3008 = curr_3008.next_3008;
    }
    System.out.println();
}
```

Gambar 2.2.2.4 cetak list

Fungsi ini digunakan untuk menampilkan/mencetak isi list dengan tanda hubung <->.

5. Membuat dan mencetak DLL awal

```
//membuat dll 2 <-> 3 <-> 5
NodeDLL_2511533008 head_3008 = new NodeDLL_2511533008(2);
head_3008.next_3008 = new NodeDLL_2511533008(3);
head_3008.next_3008.prev_3008 = head_3008;
head_3008.next_3008.next_3008 = new NodeDLL_2511533008(5);
head_3008.next_3008.next_3008.prev_3008 = head_3008.next_3008;
//cetak dll awal
System.out.print("DLL awal: ");
printList_3008(head_3008);
```

Gambar 2.2.2.5

6. Menambahkan 1 pada awal list, menambahkan 6 pada akhir list dan menambahkan 4 pada posisi 4.

```

//tambah 1 diawal
head_3008 = insertBegin_3008(head_3008, 1);
System.out.print("simpul 1 ditambah di awal: ");
printList_3008(head_3008);
//tambah 6 diakhir
System.out.print("simpul 6 ditambah di akhir: ");
int data_3008 = 6;
head_3008 = insertEnd_3008(head_3008, data_3008);
printList_3008(head_3008);
//menambah node 4 di posisi 4
System.out.print("tambah node 4 di posisi 4: ");
int data2_3008 = 4;
int pos_3008 = 4;
head_3008 = insertAtPosition_3008(head_3008, pos_3008, data2_3008);
printList_3008(head_3008);

```

Gambar 2.2.2.6 tambah di awal, akhir, posisi 4

2.2.3 Class HapusDLL_2511533008

Kode Program:

```

package pekan6_2511533008;

public class HapusDLL_2511533008 {
    //fungsi menghapus node di awal
    public static NodeDLL_2511533008
delHead_3008(NodeDLL_2511533008 head_3008) {
        if (head_3008 == null) {
            return null;
        }
        head_3008 = head_3008.next_3008;
        if (head_3008 != null) {
            head_3008.prev_3008 = null;
        }return head_3008;
    }
    //fungsi menghapus diakhir
    public static NodeDLL_2511533008
delLast_3008(NodeDLL_2511533008 head_3008) {
        if (head_3008 == null) {
            return null;
        } if (head_3008.next_3008 == null) {
            return null;
        } NodeDLL_2511533008 curr_3008 = head_3008;
        while (curr_3008.next_3008 != null) {
            curr_3008 = curr_3008.next_3008;
        }
        //update pointer previous node
        if (curr_3008.prev_3008 != null) {
            curr_3008.prev_3008.next_3008 = null;
        }return head_3008;
    }
    //fungsi menghapus node di posisi tertentu
    public static NodeDLL_2511533008
delPos_3008(NodeDLL_2511533008 head_3008, int pos_3008) {
        //jika dll kosong

```

```

        if (head_3008 == null) {
            return head_3008;
        } NodeDLL_2511533008 curr_3008 = head_3008;
        //telusuri sampai ke node yang akan dihapus
        for (int i_3008 = 1; curr_3008 != null && i_3008 <
pos_3008; ++i_3008) {
            curr_3008 = curr_3008.next_3008;
        } //jika posisi tidak ditemukan
        if (curr_3008 == null) {
            return head_3008;
        } //update pointer
        if (curr_3008.prev_3008 != null) {
            curr_3008.prev_3008.next_3008 =
curr_3008.next_3008;
        } if (curr_3008.next_3008 != null) {
            curr_3008.next_3008.prev_3008 =
curr_3008.prev_3008;
        } //jika yang di hapus head
        if (head_3008 == curr_3008) {
            head_3008 = curr_3008.next_3008;
        } return head_3008;
    }
    //fungsi mencetak DLL
    public static void printList_3008(NodeDLL_2511533008
head_3008) {
        NodeDLL_2511533008 curr_3008 = head_3008;
        while (curr_3008 != null) {
            System.out.print(curr_3008.data_3008 + " <-> ");
            curr_3008 = curr_3008.next_3008;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        //buat sebuah dll
        NodeDLL_2511533008 head_3008 = new
NodeDLL_2511533008(1);
        head_3008.next_3008 = new NodeDLL_2511533008(2);
        head_3008.next_3008.prev_3008 = head_3008;
        head_3008.next_3008.next_3008 = new
NodeDLL_2511533008(3);
        head_3008.next_3008.next_3008.prev_3008 =
head_3008.next_3008;
        head_3008.next_3008.next_3008.next_3008 = new
NodeDLL_2511533008(4);
        head_3008.next_3008.next_3008.next_3008.prev_3008 =
head_3008.next_3008.next_3008;
        head_3008.next_3008.next_3008.next_3008.next_3008 = new
NodeDLL_2511533008(5);

        head_3008.next_3008.next_3008.next_3008.next_3008.prev_3008 =
head_3008.next_3008.next_3008.next_3008;
    }

```

```

        System.out.print("DLL Awal: ");
        printList_3008(head_3008);

        System.out.print("Setelah head dihapus: ");
        head_3008 = delHead_3008(head_3008);
        printList_3008(head_3008);

        System.out.print("Setelah node terakhir dihapus: ");
        head_3008 = dellLast_3008(head_3008);
        printList_3008(head_3008);

        System.out.print("Setelah node ke 2 dihapus: ");
        head_3008 = delPos_3008(head_3008, 2);
        printList_3008(head_3008);

    }
}

```

Penjelasan:

1. Fungsi menghapus node di awal

```

//fungsi menghapus node di awal
public static NodeDLL_2511533008 delHead_3008(NodeDLL_2511533008 head_3008) {
    if (head_3008 == null) {
        return null;
    }
    head_3008 = head_3008.next_3008;
    if (head_3008 != null) {
        head_3008.prev_3008 = null;
    }
    return head_3008;
}

```

Gunakan 2.2.3.1 hapus head

Jika list kosong maka return null, jika ada node maka pindahkan head ke node berikutnya. Putuskan hubungan prev dari head baru agar tidak menunjuk ke node lama.

2. Fungsi menghapus node di akhir

```

//fungsi menghapus diakhir
public static NodeDLL_2511533008 dellLast_3008(NodeDLL_2511533008 head_3008) {
    if (head_3008 == null) {
        return null;
    }
    if (head_3008.next_3008 == null) {
        return null;
    }
    NodeDLL_2511533008 curr_3008 = head_3008;
    while (curr_3008.next_3008 != null) {
        curr_3008 = curr_3008.next_3008;
    }
    //update pointer previous node
    if (curr_3008.prev_3008 != null) {
        curr_3008.prev_3008.next_3008 = null;
    }
    return head_3008;
}

```

Gambar 2.2.3.2 hapus node akhir

Jika list kosong maka return null, jika hanya ada satu node maka return null (list jadi kosong). Lalu traversal sampai node terakhir dan putuskan hubungan node terakhir dengan node sebelumnya.

3. Fungsi hapus node di posisi tertentu

```
//fungsi menghapus node di posisi tertentu
public static NodeDLL_2511533008 delPos_3008(NodeDLL_2511533008 head_3008, int pos_3008) {
    //jika dll kosong
    if (head_3008 == null) {
        return head_3008;
    }
    NodeDLL_2511533008 curr_3008 = head_3008;
    //telusuri sampai ke node yang akan dihapus
    for (int i_3008 = 1; curr_3008 != null && i_3008 < pos_3008; ++i_3008) {
        curr_3008 = curr_3008.next_3008;
    }
    //jika posisi tidak ditemukan
    if (curr_3008 == null) {
        return head_3008;
    }
    //update pointer
    if (curr_3008.prev_3008 != null) {
        curr_3008.prev_3008.next_3008 = curr_3008.next_3008;
    }
    if (curr_3008.next_3008 != null) {
        curr_3008.next_3008.prev_3008 = curr_3008.prev_3008;
    }
    //jika yang di hapus head
    if (head_3008 == curr_3008) {
        head_3008 = curr_3008.next_3008;
    }
    return head_3008;
}
```

Gambar 2.2.3.3 hapus node di posisi tertentu

Traversal sampai posisi yang diminta. Jika posisi tidak ada maka return head. Update pointer prev dan next agar node yang dihapus tidak lagi terhubung.

4. Fungsi cetak list

```
//fungsi mencetak DLL
public static void printList_3008(NodeDLL_2511533008 head_3008) {
    NodeDLL_2511533008 curr_3008 = head_3008;
    while (curr_3008 != null) {
        System.out.print(curr_3008.data_3008 + " <-> ");
        curr_3008 = curr_3008.next_3008;
    }
    System.out.println();
}
```

Gambar 2.2.3.4 cetak list

Fungsi ini digunakan untuk menampilkan/mencetak isi list dengan tanda hubung <->.

5. Membuat dan mencetak DLL awal

```
//buat sebuah dll
NodeDLL_2511533008 head_3008 = new NodeDLL_2511533008(1);
head_3008.next_3008 = new NodeDLL_2511533008(2);
head_3008.next_3008.prev_3008 = head_3008;
head_3008.next_3008.next_3008 = new NodeDLL_2511533008(3);
head_3008.next_3008.next_3008.prev_3008 = head_3008.next_3008;
head_3008.next_3008.next_3008.next_3008 = new NodeDLL_2511533008(4);
head_3008.next_3008.next_3008.next_3008.prev_3008 = head_3008.next_3008.next_3008;
head_3008.next_3008.next_3008.next_3008.next_3008 = new NodeDLL_2511533008(5);
head_3008.next_3008.next_3008.next_3008.next_3008.prev_3008 = head_3008.next_3008.next_3008.next_3008;

System.out.print("DLL Awal: ");
printList_3008(head_3008);
```

Gambar 2.2.3.5 DLL awal

6. Hapus head, lalu hapus node terakhir dan hapus node posisi 2.

```
System.out.print("Setelah head dihapus: ");
head_3008 = delHead_3008(head_3008);
printList_3008(head_3008);

System.out.print("Setelah node terakhir dihapus: ");
head_3008 = delLast_3008(head_3008);
printList_3008(head_3008);

System.out.print("Setelah node ke 2 dihapus: ");
head_3008 = delPos_3008(head_3008, 2);
printList_3008(head_3008);
```

Gambar 2.2.3.6 hapus node pada DLL

2.2.4 Class PenelusuranDLL_2511533008

Kode program:

```
package pekan6_2511533008;

public class PenelusuranDLL_2511533008 {
    //fungsi penelusuran
    static void forwardTraversal_3008(NodeDLL_2511533008
head_3008) {
        //memulai penelusuran dari head
        NodeDLL_2511533008 curr_3008 = head_3008;
        //lanjutkan sampai akhir
        while (curr_3008 != null) {
            //print data
            System.out.print(curr_3008.data_3008 + " <-> ");
            //pindah ke node berikutnya
            curr_3008 = curr_3008.next_3008;
        }
        //print spasi
        System.out.println();
    }
    //fungsi penelusuran mundur
    static void backwardTraversal_3008(NodeDLL_2511533008
tail_3008) {
        //mulai dari akhir
        NodeDLL_2511533008 curr_3008 = tail_3008;
        //lanjut sampai head
```



```

        while (curr_3008 != null) {
            //cetak data
            System.out.print(curr_3008.data_3008 + " <-> ");
            //pindah ke node sebelumnya
            curr_3008 = curr_3008.prev_3008;
        }
        //cetak spasi
        System.out.println();
    }

    public static void main(String[] args) {
        //cetak dll
        NodeDLL_2511533008 head_3008 = new
NodeDLL_2511533008(1);
        NodeDLL_2511533008 second_3008 = new
NodeDLL_2511533008(2);
        NodeDLL_2511533008 third_3008 = new
NodeDLL_2511533008(3);

        head_3008.next_3008 = second_3008;
        second_3008.prev_3008 = head_3008;
        second_3008.next_3008 = third_3008;
        third_3008.prev_3008 = second_3008;

        System.out.println("Penelusuran maju: ");
        forwardTraversal_3008(head_3008);

        System.out.println("Penelusuran mundur: ");
        backwardTraversal_3008(third_3008);
    }
}

```

Penjelasan:

1. Fungsi penelusuran maju

```

//fungsi penelusuran
static void forwardTraversal_3008(NodeDLL_2511533008 head_3008) {
    //memulai penelusuran dari head
    NodeDLL_2511533008 curr_3008 = head_3008;
    //lanjutkan sampai akhir
    while (curr_3008 != null) {
        //print data
        System.out.print(curr_3008.data_3008 + " <-> ");
        //pindah ke node berikutnya
        curr_3008 = curr_3008.next_3008;
    }
    //print spasi
    System.out.println();
}

```

Gambar 2.2.4.1 penelusuran maju

Mulai dari head, selama node tidak null, cetak data. Lanjut ke node berikutnya (next). Hasilnya traversal dari depan ke belakang.

2. Fungsi penelusuran mundur

```
//fungsi penelusuran mundur
static void backwardTraversal_3008(NodeDLL_2511533008 tail_3008) {
    //mulai dari akhir
    NodeDLL_2511533008 curr_3008 = tail_3008;
    //lanjut sampai head
    while (curr_3008 != null) {
        //cetak data
        System.out.print(curr_3008.data_3008 + " <-> ");
        //pindah ke node sebelumnya
        curr_3008 = curr_3008.prev_3008;
    }
    //cetak spasi
    System.out.println();
}
```

Gambar 2.2.4.2 penelusuran mundur

Mulai dari tail, selama node tidak null, cetak data. Lanjut ke node sebelumnya (prev). Hasilnya traversal dari belakang ke depan.

3. Fungsi utama

```
public static void main(String[] args) {
    //cetak dll
    NodeDLL_2511533008 head_3008 = new NodeDLL_2511533008(1);
    NodeDLL_2511533008 second_3008 = new NodeDLL_2511533008(2);
    NodeDLL_2511533008 third_3008 = new NodeDLL_2511533008(3);

    head_3008.next_3008 = second_3008;
    second_3008.prev_3008 = head_3008;
    second_3008.next_3008 = third_3008;
    third_3008.prev_3008 = second_3008;

    System.out.println("Penelusuran maju: ");
    forwardTraversal_3008(head_3008);

    System.out.println("Penelusuran mundur: ");
    backwardTraversal_3008(third_3008);
}
```

Gambar 2.2.4.3 fungsi utama

Membuat DLL sederhana lalu melakukan traversal maju dan traversal mundur.

2.3 Output Program

2.3.1 Output InsertDLL_2511533008

```
<terminated> InsertDLL_2511533008 [Java Application] C:\Users\asusvivobook\p2\pool\plugi
DLL awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

Gambar 2.3.1 output insert dll

2.3.2 Output HapusDLL_2511533008

```
<terminated> HapusDLL_2511533008 [Java Application] C:\Users\asusvivoboc
DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->
Setelah node ke 2 dihapus: 2 <-> 4 <->
```

Gambar 2.3.2 output hapus dll

2.3.3 Output PenelusuranDLL_2511533008

```
<terminated> PenelusuranDLL_251153
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

Gambar 2.3.3 punelusuran dll

BAB III

PENUTUP

3.1 Kesimpulan

Kesimpulan dari praktikum struktur data Double Linked List adalah bahwa DLL merupakan salah satu bentuk pengembangan dari Single Linked List yang memiliki dua pointer, yaitu next dan prev, sehingga memungkinkan penelusuran dilakukan dari dua arah sekaligus. Melalui praktikum ini, kita dapat memahami bagaimana cara membuat node, melakukan operasi penyisipan (insert) di awal, akhir, maupun posisi tertentu, serta operasi penghapusan (delete) pada berbagai posisi dalam list. Selain itu, penelusuran maju dan mundur memberikan fleksibilitas dalam mengakses data, yang tidak dimiliki oleh Single Linked List. Dengan menguasai konsep DLL, kita memperoleh dasar penting untuk mempelajari struktur data yang lebih kompleks seperti tree dan graph, sekaligus meningkatkan keterampilan logika pemrograman dan efisiensi pengolahan data dalam pengembangan perangkat lunak.

DAFTAR PUSTAKA

- [1] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Wiley, 2014.
- [2] I. Lim, *Diktat Struktur Data*. Bandung: Institut Teknologi Bandung, 2008.
- [3] Universitas Andalas, "Linked List," Fakultas Teknologi Informasi, 2026.

TUGAS

1. Kode Program

1.1 Class Lagu_2511533008

```
package pekan6_2511533008;

public class Lagu_2511533008 {
    //mendefinisikan kelas lagu
    String judul_3008;
    String penyanyi_3008;
    Lagu_2511533008 next_3008; //pointer ke next node
    Lagu_2511533008 prev_3008; //pointer ke previous node

    //konstruktor
    public Lagu_2511533008(String judul_3008, String
    penyanyi_3008) {
        this.judul_3008 = judul_3008;
        this.penyanyi_3008 = penyanyi_3008;
        this.next_3008 = null;
        this.prev_3008 = null;
    }

    //getter
    public String getJudul_3008() {return judul_3008;}
    public String getPenyanyi_3008() {return penyanyi_3008;}

    //setter
    public void setJudul_3008(String judul_3008) {this.judul_3008
    = judul_3008;}
    public void setPenyanyi_3008(String penyanyi_3008)
    {this.penyanyi_3008 = penyanyi_3008;}
}
```

1.2 Class Musik_2511533008

```
package pekan6_2511533008;
import java.util.Scanner;

public class Musik_2511533008 {
    // tambah lagu di akhir
    static Lagu_2511533008 tambahLagu_3008(Lagu_2511533008 head_3008,
    String judul_3008, String penyanyi_3008) {
        Lagu_2511533008 newLagu_3008 = new
    Lagu_2511533008(judul_3008, penyanyi_3008);
        if (head_3008 == null) {
            head_3008 = newLagu_3008;
        } else {
            Lagu_2511533008 curr_3008 = head_3008;
            while (curr_3008.next_3008 != null) {
                curr_3008 = curr_3008.next_3008;
            }
            curr_3008.next_3008 = newLagu_3008;
        }
        return head_3008;
    }
}
```

```

    }
    curr_3008.next_3008 = newLagu_3008;
    newLagu_3008.prev_3008 = curr_3008;
}
return head_3008;
}

// hapus lagu pertama
public static Lagu_2511533008 hapusLaguAwal_3008(Lagu_2511533008
head_3008) {
    if (head_3008 == null) return null;
    head_3008 = head_3008.next_3008;
    if (head_3008 != null) head_3008.prev_3008 = null;
    return head_3008;
}

// tampil maju
static void tampilMaju_3008(Lagu_2511533008 head_3008) {
    Lagu_2511533008 curr_3008 = head_3008;
    while (curr_3008 != null) {
        System.out.print(curr_3008.judul_3008 + " - " +
curr_3008.penyanyi_3008 + " | ");
        curr_3008 = curr_3008.next_3008;
    }
    System.out.println();
}

// tampil mundur
static void tampilMundur_3008(Lagu_2511533008 head_3008) {
    if (head_3008 == null) return;
    Lagu_2511533008 curr_3008 = head_3008;
    while (curr_3008.next_3008 != null) {
        curr_3008 = curr_3008.next_3008;
    }
    while (curr_3008 != null) {
        System.out.print(curr_3008.judul_3008 + " - " +
curr_3008.penyanyi_3008 + " | ");
        curr_3008 = curr_3008.prev_3008;
    }
    System.out.println();
}

// cari lagu
public void cariLagu_3008(Lagu_2511533008 head_3008, String
judul_3008) {
    Lagu_2511533008 curr_3008 = head_3008;
    while (curr_3008 != null) {
        if (curr_3008.judul_3008.equalsIgnoreCase(judul_3008)) {
            System.out.println("Lagu ditemukan: " +
curr_3008.judul_3008 + " - " + curr_3008.penyanyi_3008);
            return;
        }
        curr_3008 = curr_3008.next_3008;
    }
}

```

```

    }
    System.out.println("Lagu tidak ditemukan!");
}

// menu
public static void tampilkanPilihan_3008() {
    System.out.println("=== Playlist Musik NIM: 2511533008 ===");
    System.out.println("1. Tambah Lagu");
    System.out.println("2. Hapus Lagu Pertama");
    System.out.println("3. Lihat Playlist (Maju)");
    System.out.println("4. Lihat Playlist (Mundur)");
    System.out.println("5. Cari Lagu");
    System.out.println("6. Keluar");
}

public static void main(String[] args) {
    Scanner sc_3008 = new Scanner(System.in);
    Musik_2511533008 playlist_3008 = new Musik_2511533008();
    Lagu_2511533008 head_3008 = null;
    int chose_3008;

    do {
        tampilkanPilihan_3008();
        System.out.print("Pilihan : ");
        chose_3008 = sc_3008.nextInt();
        sc_3008.nextLine();

        switch (chose_3008) {
            case 1:
                System.out.print("Judul lagu: ");
                String judul_3008 = sc_3008.nextLine();
                System.out.print("Penyanyi: ");
                String penyanyi_3008 = sc_3008.nextLine();
                head_3008 = tambahLagu_3008(head_3008,
judul_3008, penyanyi_3008);
                System.out.println("Lagu berhasil ditambahkan");
                break;
            case 2:
                head_3008 = hapusLaguAwal_3008(head_3008);
                System.out.println("Lagu pertama berhasil
dihapus");
                break;
            case 3:
                tampilMaju_3008(head_3008);
                break;
            case 4:
                tampilMundur_3008(head_3008);
                break;
            case 5:
                System.out.print("Masukkan judul lagu yang
dicari: ");
                String cari_3008 = sc_3008.nextLine();

```



```

        playlist_3008.cariLagu_3008(head_3008,
cari_3008);
        break;
    case 6:
        System.out.println("Keluar dari program.");
        break;
    default:
        System.out.println("Pilihan tidak valid");
    }
} while (choise_3008 != 6);

sc_3008.close();
}
}

```

2. Output Tugas

```

=== Playlist Musik NIM: 2511533008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 1
Judul lagu: sallou
Penyanyi: maher zain
Lagu berhasil ditambahkan
=== Playlist Musik NIM: 2511533008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 1
Judul lagu: nas tehbehlana
Penyanyi: maher zain
Lagu berhasil ditambahkan
=== Playlist Musik NIM: 2511533008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 3
sallou - maher zain | nas tehbehlana - maher zain |

```

```

=== Playlist Musik NIM: 2511533008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 4
nas tehbehlana - maher zain | sallou - maher zain |
=== Playlist Musik NIM: 2511533008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 5
Masukkan judul lagu yang dicari: sallou
Lagu ditemukan: sallou - maher zain
=== Playlist Musik NIM: 2511533008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 2
Lagu pertama berhasil dihapus

```

```

=== Playlist Musik NIM: 2511533008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 3
nas tehbehlana - maher zain |
=== Playlist Musik NIM: 2511533008 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan : 6
Keluar dari program.

```